and no more referential-integrity problems.

NOTE The main exception I've found to this pattern is when you're storing your users' personally identifying information. In these cases, you may be duty-bound (and potentially legally bound) to scrub their information from your database on request.

With this approach, you can create a delete method on `RecipeService` that updates the `IsDeleted` flag, as shown in listing 12.10. In addition, make sure that you have `Where()` clauses in all the other methods in your `RecipeService` to ensure you can't return a deleted `Recipe`, as you saw in listing 12.9 for the `GetRecipes()` method.

Listing 12.10 Marking entities as deleted in EF Core

```
public async Task DeleteRecipe(int recipeId)
{
    var recipe = await _context.Recipes.FindAsync(recipeId);  ❶
    if(recipe is null) {                                      ❷
        throw new Exception("Unable to find the recipe");     ❷
    }                                                         ❷
    recipe.IsDeleted = true;                                  ❸
    await _context.SaveChangesAsync();                        ❹
}
```

❶ Fetches the Recipe entity by id

❷ If an invalid id is provided, recipe will be null.

❸ Marks the Recipe as deleted

❹ Executes the SQL to save the changes to the database

This approach satisfies the requirements—it removes the recipe from exposure by the API—but it simplifies several things. This soft-delete approach won't work for all scenarios, but I've found it to be a common pattern in projects I've worked on.